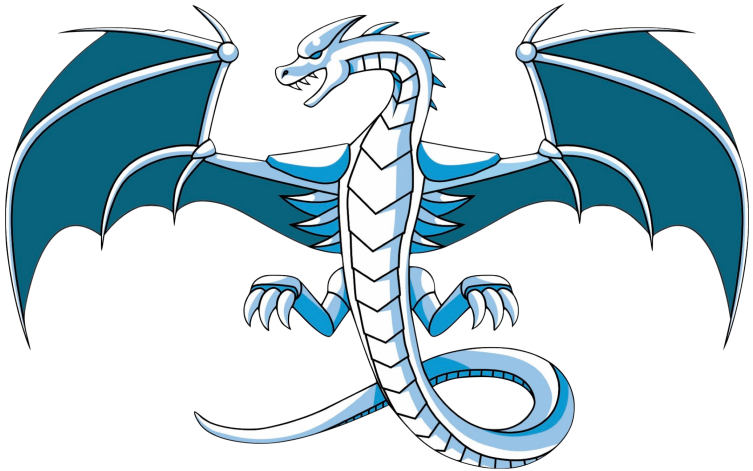
LLVM Obfuscation



For the rest of Us

Schedule

We will “try” to comply with it :)

Morning

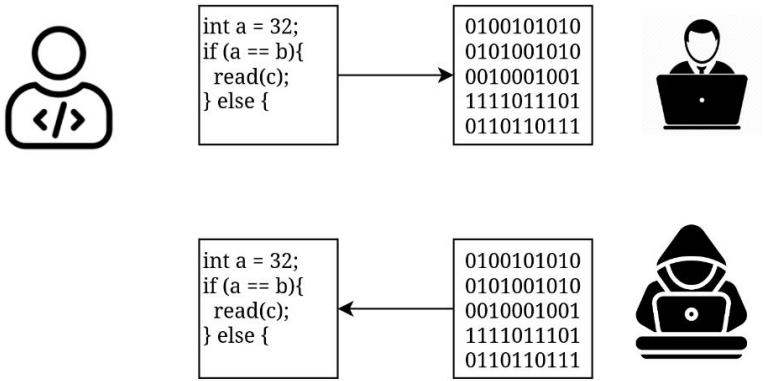
- Why obfuscation
- What is LLVM
- Hello World in LLVM
- List function names
- List functions
- List Basic Blocks
- List Instructions

Afternoon

- Basic program modification
- Arithmetic Obfuscation
- Basic Block Splitting
- Control Flow Flattening
- Making a pass pipeline

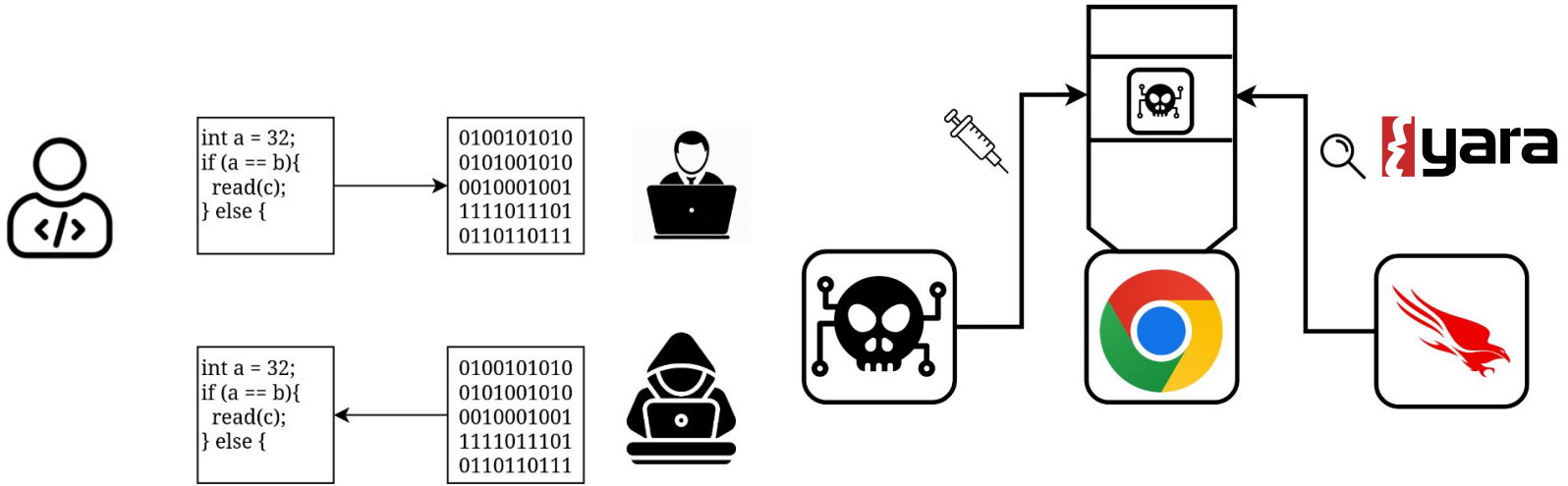
Why obfuscation

For security purposes ofc



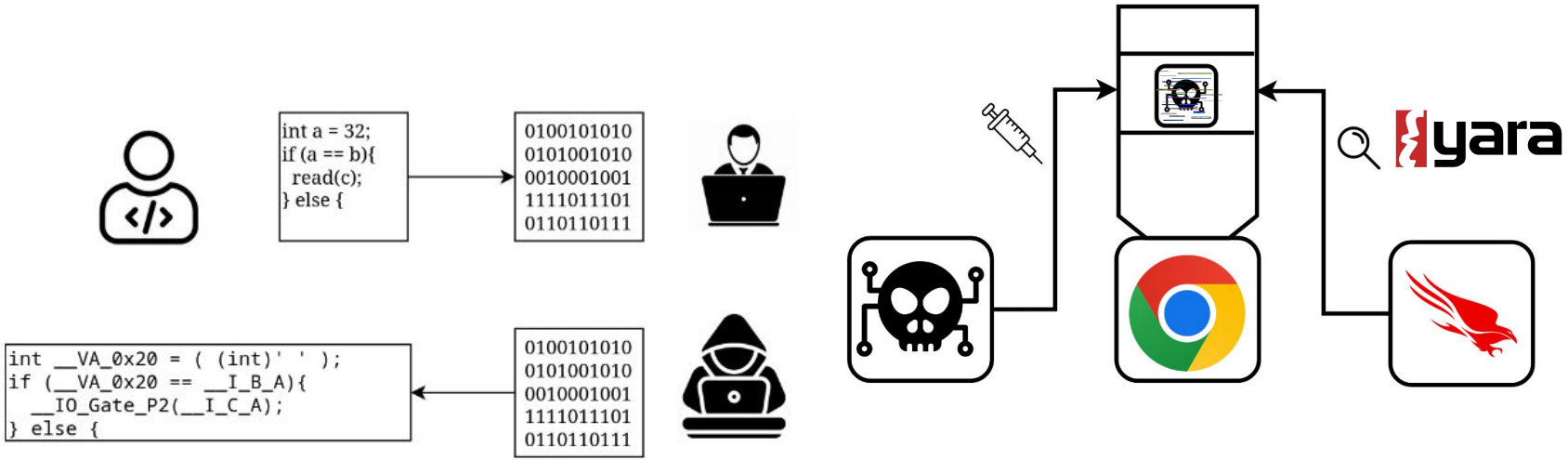
Why obfuscation

For security purposes ofc



Why obfuscation

For security purposes ofc



What obfuscation

Can’t we just use a tool?

What obfuscation

Can’t we just use a tool?

What obfuscation

Can’t we just use a tool?

- Paid ($$$)
- Absolutely proprietary
- Windows only

What obfuscation

Can’t we just use a tool?

- Paid ($$$)
- Absolutely proprietary
- Windows only

What obfuscation

Can’t we just use a tool?

- Paid ($$$)
- Absolutely proprietary
- Windows only

- C -> C

What obfuscation

Can't we just use a tool?

- Paid ($$$)
- Absolutely proprietary
- Windows only

- C -> C

What obfuscation

Can't we just use a tool?

- Paid ($$$)
- Absolutely proprietary
- Windows only

- C -> C

- Just a packer (lol)

What obfuscation

Can't we just use a tool?

- Paid ($$$)
- Absolutely proprietary
- Windows only

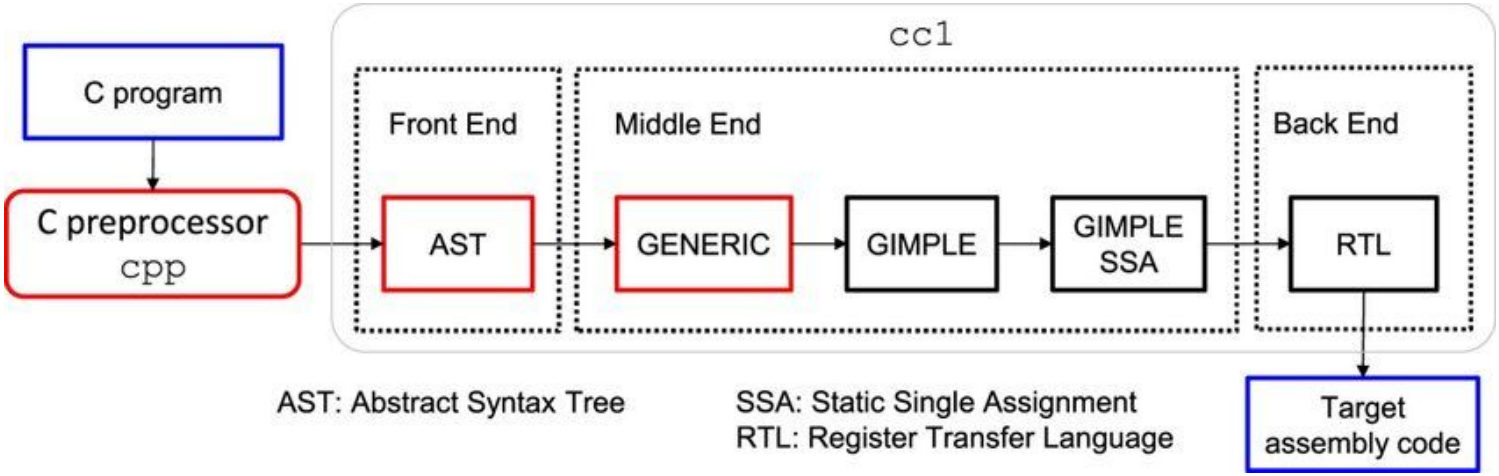
- C -> C

- Just a packer (lol)

- We want to run shellcode
- We want malleability
- We want flexibility

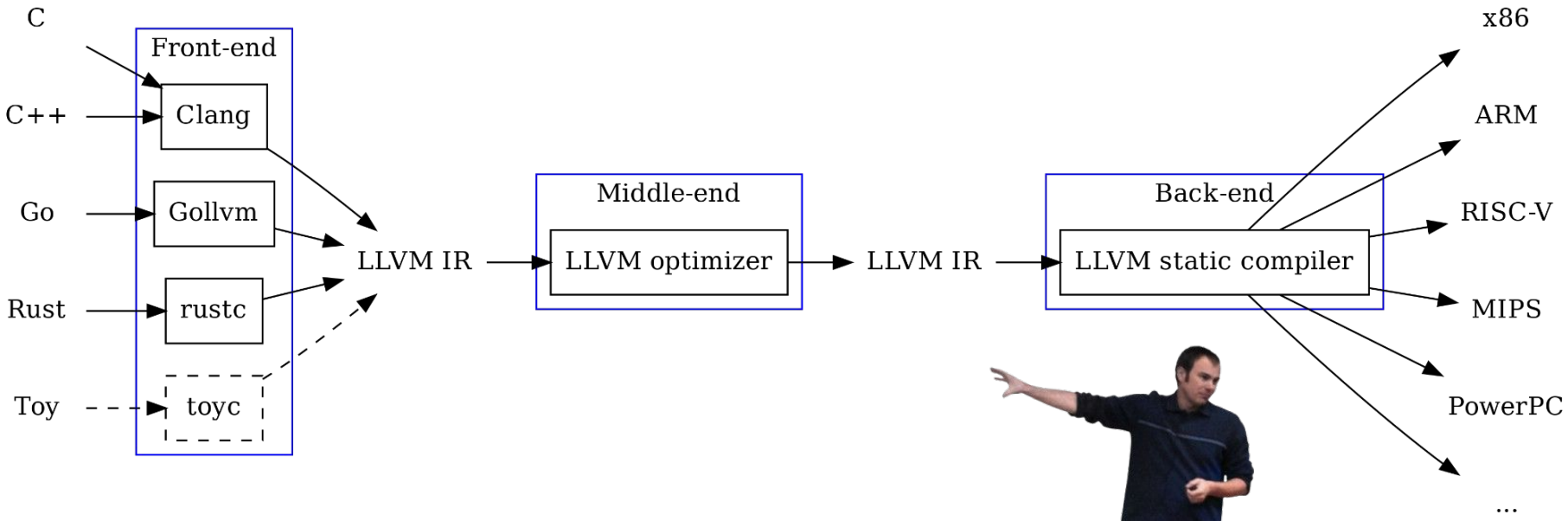
What is LLVM

And why it is so important



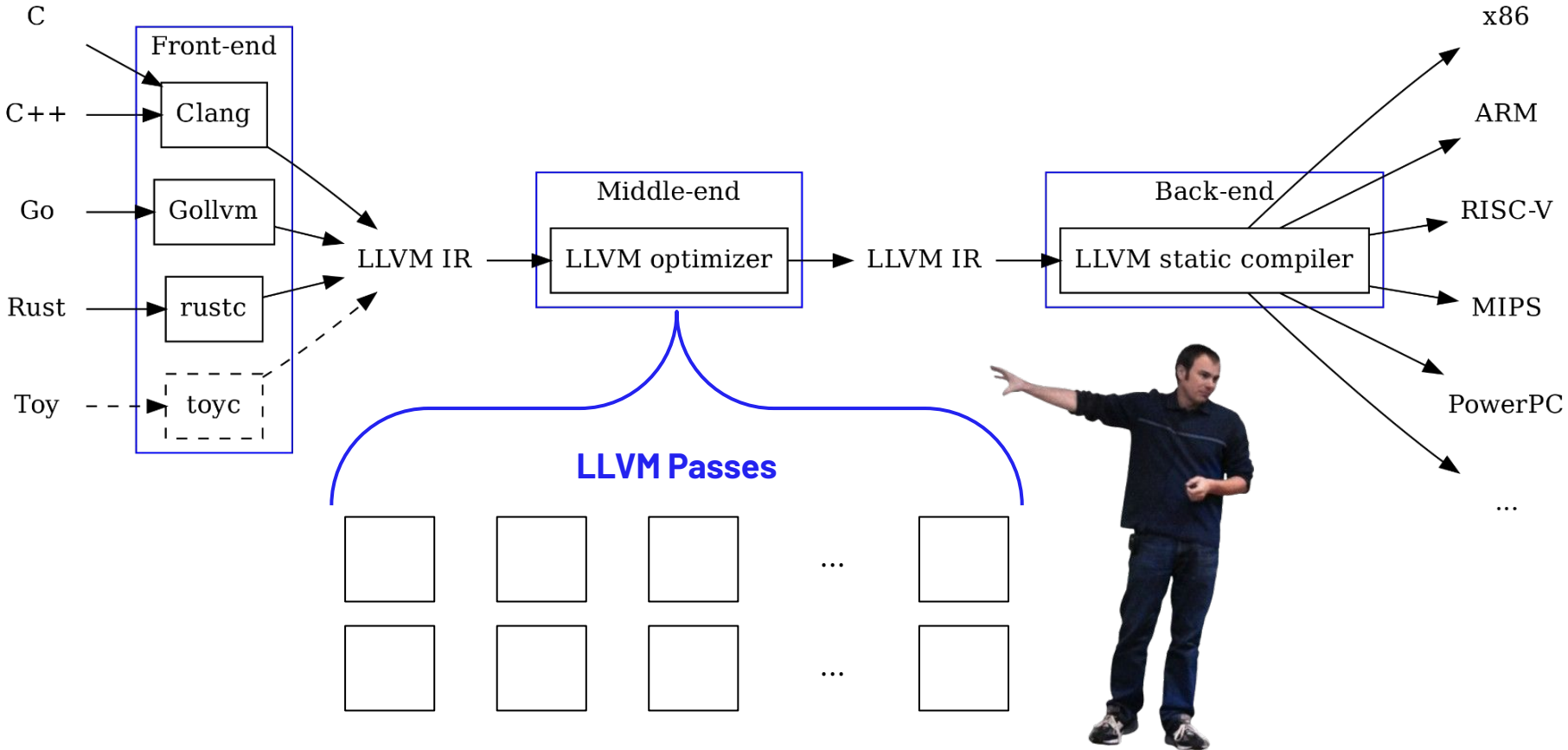
What is LLVM

And why it is so important



What is LLVM

And why it is so important



Hello World

The first steps

Hello World

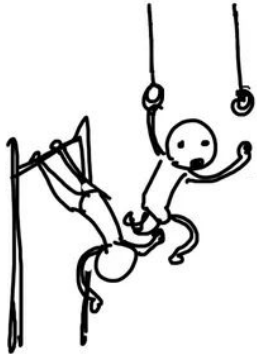
The first steps

```
$ sudo apt install llvm-17-dev libclang-17-dev
```

Hello World

The first steps

```
$ sudo apt install llvm-17-dev libclang-17-dev
```

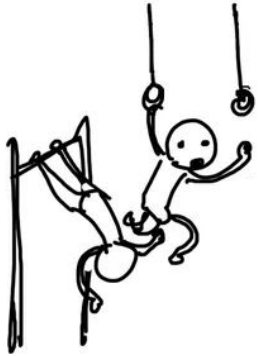


```
$ mkdir hello-world
$ cd hello-world
$ vim CMakeLists.txt
$ vim main.c
$ mkdir build
$ cd build
$ cmake ..
$ make
```

Hello World

The first steps

```
$ sudo apt install llvm-17-dev libclang-17-dev
```



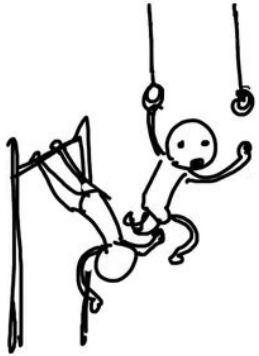
```
$ clang -S -emit-llvm test.c -o test.ll
```

```
$ mkdir hello-world
$ cd hello-world
$ vim CMakeLists.txt
$ vim main.c
$ mkdir build
$ cd build
$ cmake ..
$ make
```

Hello World

The first steps

```
$ sudo apt install llvm-17-dev libclang-17-dev
```



```
$ mkdir hello-world
$ cd hello-world
$ vim CMakeLists.txt
$ vim main.c
$ mkdir build
$ cd build
$ cmake ..
$ make
```

```
$ clang -S -emit-llvm test.c -o test.ll
```

```
$ opt -S \
  -load-pass-plugin=./libHelloPass.so \
  -passes="hello-world" \
  test.ll -o /dev/null
```

Hello World

But now without the hassle

Hello World

But now without the hassle

```
$ make pod-build
```

Hello World

But now without the hassle

```
$ make pod-build
```

```
$ make <pass-name>
```

Hello World

But now without the hassle

```
$ make pod-build
```

```
$ make <pass-name>
```

```
$ make test
```

Hello World

But now without the hassle

```cpp
#include "llvm/Pass.h"
#include "llvm/IR/Module.h"
#include "llvm/Passes/PassBuilder.h"
#include "llvm/Passes/PassPlugin.h"
#include "llvm/Support/raw_ostream.h"

using namespace llvm;

namespace {
    struct HelloWorldPass : public PassInfoMixin<HelloWorldPass> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            errs() << "Hello, World!\n";
            return PreservedAnalyses::all();
        };
    };
}

extern "C" LLVM_ATTRIBUTE_WEAK PassPluginLibraryInfo llvmGetPassPluginInfo() {
    return {
        .APIVersion = LLVM_PLUGIN_API_VERSION,
        .PluginName = "HelloWorldPass",
        .PluginVersion = "v0.1",
        .RegisterPassBuilderCallbacks = [](PassBuilder &PB) {
            PB.registerPipelineStartEPCallback(
                [](ModulePassManager &MPM, OptimizationLevel Level) {
                    MPM.addPass(HelloWorldPass());
                });
        }
    };
}
```

Hello World

But now without the hassle

Execution method for a Module Pass

```cpp
#include "llvm/Pass.h"
#include "llvm/IR/Module.h"
#include "llvm/Passes/PassBuilder.h"
#include "llvm/Passes/PassPlugin.h"
#include "llvm/Support/raw_ostream.h"

using namespace llvm;

namespace {
    struct HelloWorldPass : public PassInfoMixin<HelloWorldPass> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            errs() << "Hello, World!\n";
            return PreservedAnalyses::all();
        };
    };
}

extern "C" LLVM_ATTRIBUTE_WEAK PassPluginLibraryInfo llvmGetPassPluginInfo() {
    return {
        .APIVersion = LLVM_PLUGIN_API_VERSION,
        .PluginName = "HelloWorldPass",
        .PluginVersion = "v0.1",
        .RegisterPassBuilderCallbacks = [](PassBuilder &PB) {
            PB.registerPipelineStartEPCallback(
                [](ModulePassManager &MPM, OptimizationLevel Level) {
                    MPM.addPass(HelloWorldPass());
                });
        }
    };
}
```

Hello World

But now without the hassle

Execution method for a Module Pass

Return that the pass did not modify the IR

```
#include "llvm/Pass.h"
#include "llvm/IR/Module.h"
#include "llvm/Passes/PassBuilder.h"
#include "llvm/Passes/PassPlugin.h"
#include "llvm/Support/raw_ostream.h"

using namespace llvm;

namespace {
    struct HelloWorldPass : public PassInfoMixin<HelloWorldPass> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            errs() << "Hello, World!\n";
            return PreservedAnalyses::all();
        };
    };
}

extern "C" LLVM_ATTRIBUTE_WEAK PassPluginLibraryInfo llvmGetPassPluginInfo() {
    return {
        .APIVersion = LLVM_PLUGIN_API_VERSION,
        .PluginName = "HelloWorldPass",
        .PluginVersion = "v0.1",
        .RegisterPassBuilderCallbacks = [](PassBuilder &PB) {
            PB.registerPipelineStartEPCallback(
                [](ModulePassManager &MPM, OptimizationLevel Level) {
                    MPM.addPass(HelloWorldPass());
                });
        }
    };
}
```

Hello World

But now without the hassle

Execution method for a Module Pass

Entry point function for a Pass Plugin

Return that the pass did not modify the IR

```cpp
#include "llvm/Pass.h"
#include "llvm/IR/Module.h"
#include "llvm/Passes/PassBuilder.h"
#include "llvm/Passes/PassPlugin.h"
#include "llvm/Support/raw_ostream.h"

using namespace llvm;

namespace {
    struct HelloWorldPass : public PassInfoMixin<HelloWorldPass> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            errs() << "Hello, World!\n";
            return PreservedAnalyses::all();
        };
    };
}

extern "C" LLVM_ATTRIBUTE_WEAK PassPluginLibraryInfo llvmGetPassPluginInfo() {
    return {
        .APIVersion = LLVM_PLUGIN_API_VERSION,
        .PluginName = "HelloWorldPass",
        .PluginVersion = "v0.1",
        .RegisterPassBuilderCallbacks = [](PassBuilder &PB) {
            PB.registerPipelineStartEPCallback(
                [](ModulePassManager &MPM, OptimizationLevel Level) {
                    MPM.addPass(HelloWorldPass());
                });
        }
    };
}
```

Hello World

But now without the hassle

Execution method for a Module Pass

Entry point function for a Pass Plugin

Return that the pass did not modify the IR

Register the pass in the optimization pipeline

```cpp
#include "llvm/Pass.h"
#include "llvm/IR/Module.h"
#include "llvm/Passes/PassBuilder.h"
#include "llvm/Passes/PassPlugin.h"
#include "llvm/Support/raw_ostream.h"

using namespace llvm;

namespace {
    struct HelloWorldPass : public PassInfoMixin<HelloWorldPass> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            errs() << "Hello, World!\n";
            return PreservedAnalyses::all();
        };
    };
}

extern "C" LLVM_ATTRIBUTE_WEAK PassPluginLibraryInfo llvmGetPassPluginInfo() {
    return {
        .APIVersion = LLVM_PLUGIN_API_VERSION,
        .PluginName = "HelloWorldPass",
        .PluginVersion = "v0.1",
        .RegisterPassBuilderCallbacks = [](PassBuilder &PB) {
            PB.registerPipelineStartEPCallback(
                [](ModulePassManager &MPM, OptimizationLevel Level) {
                    MPM.addPass(HelloWorldPass());
                });
        }
    };
}
```

Hello World

But now without the hassle

Into the IR

Lets go deeper

Into the IR

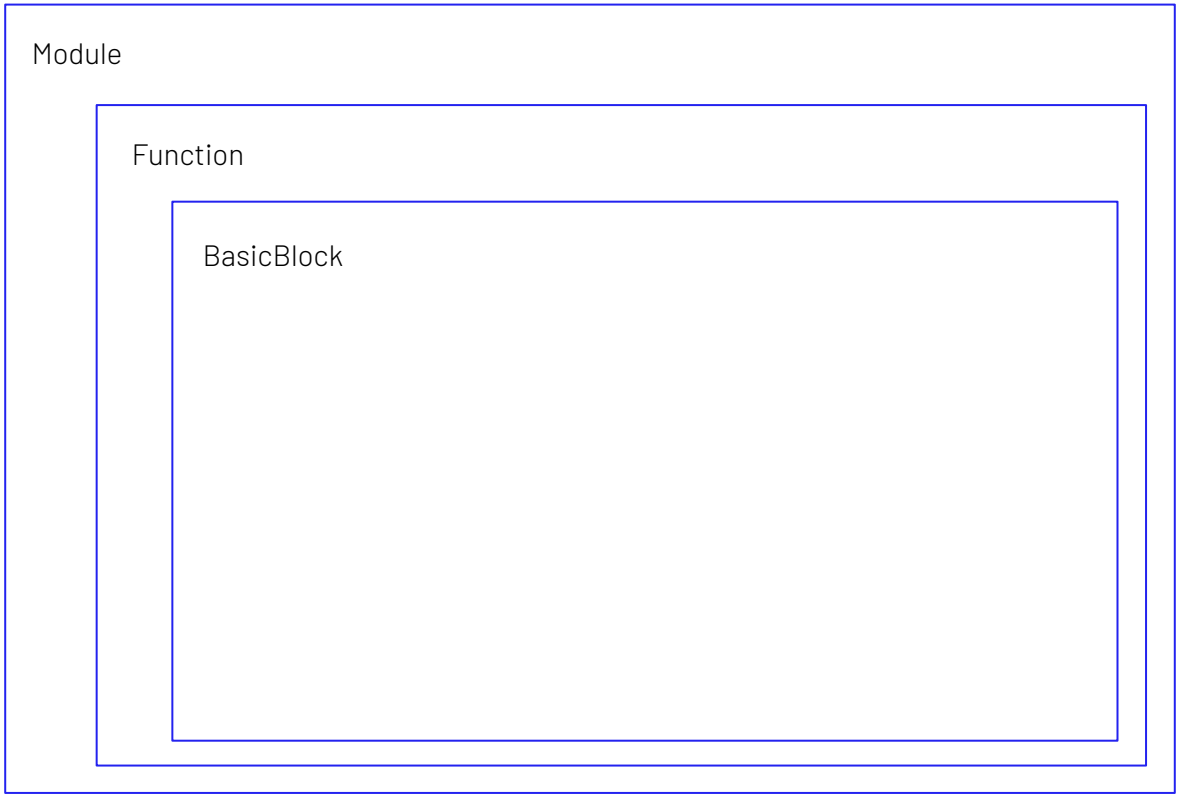
Lets go deeper

Into the IR

Lets go deeper

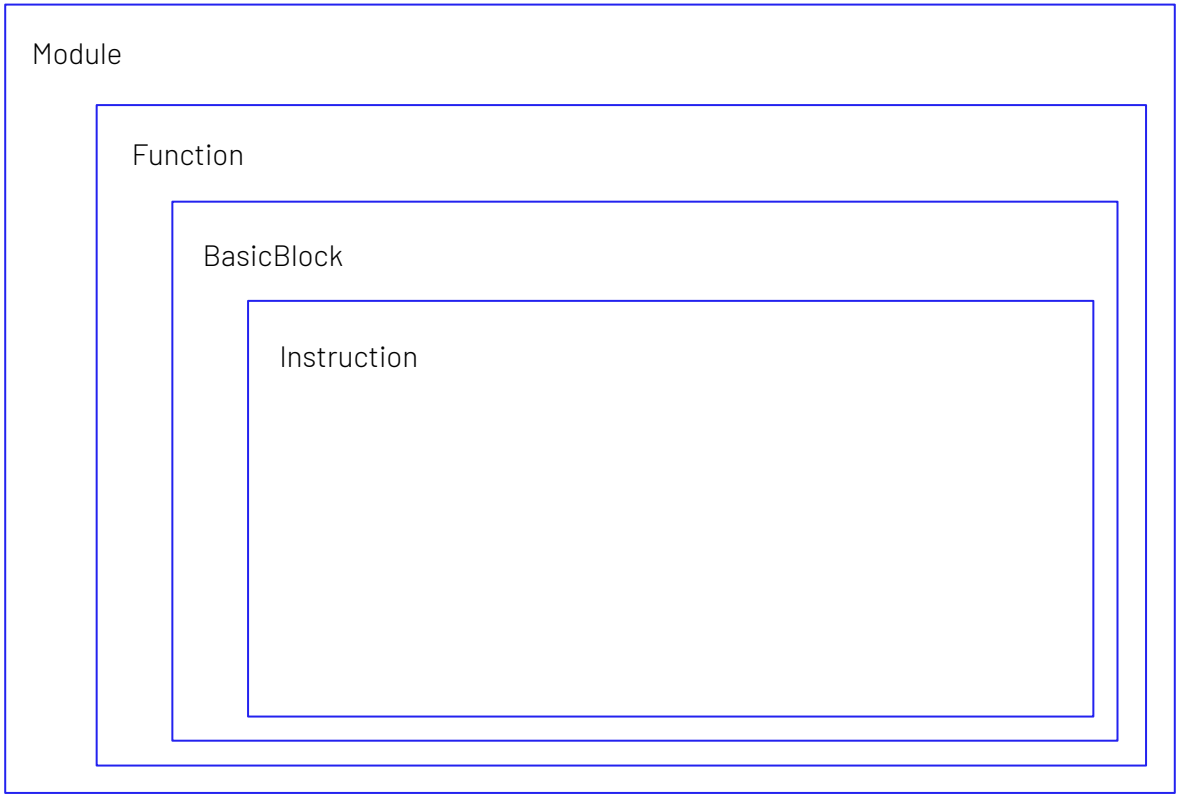
Into the IR

Lets go deeper



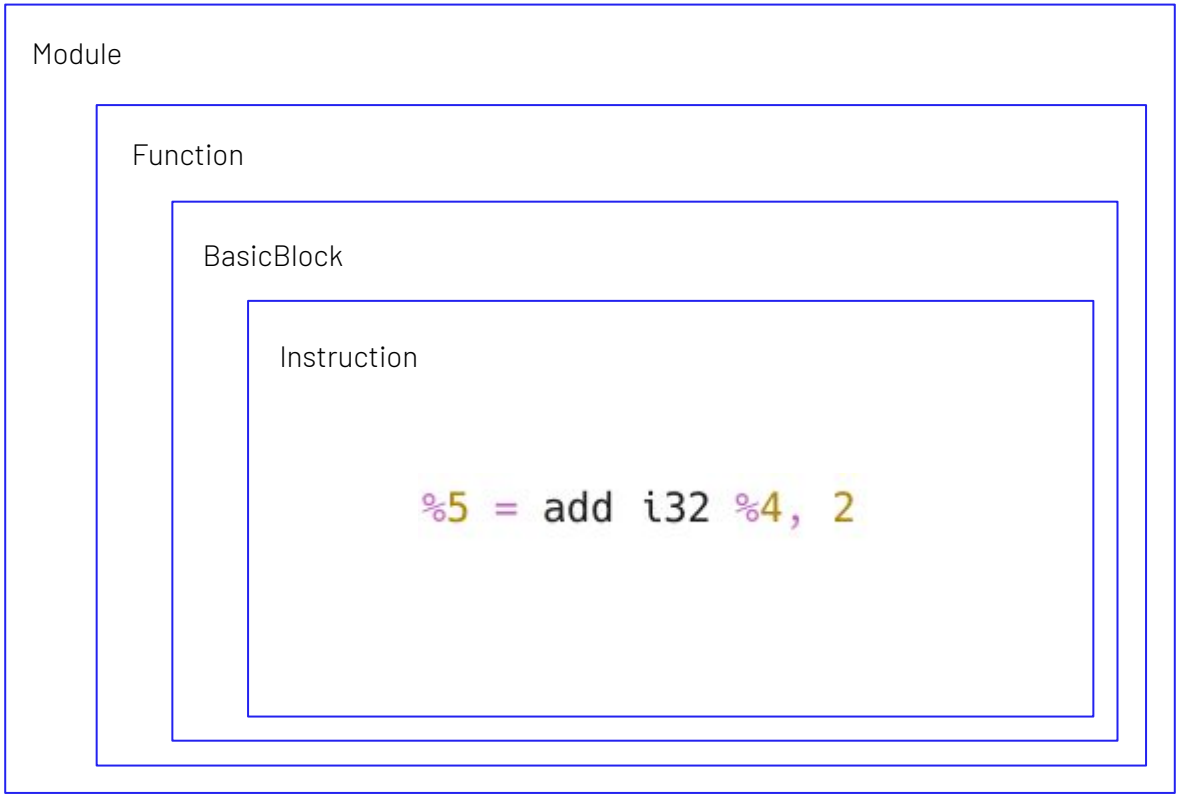
Into the IR

Lets go deeper



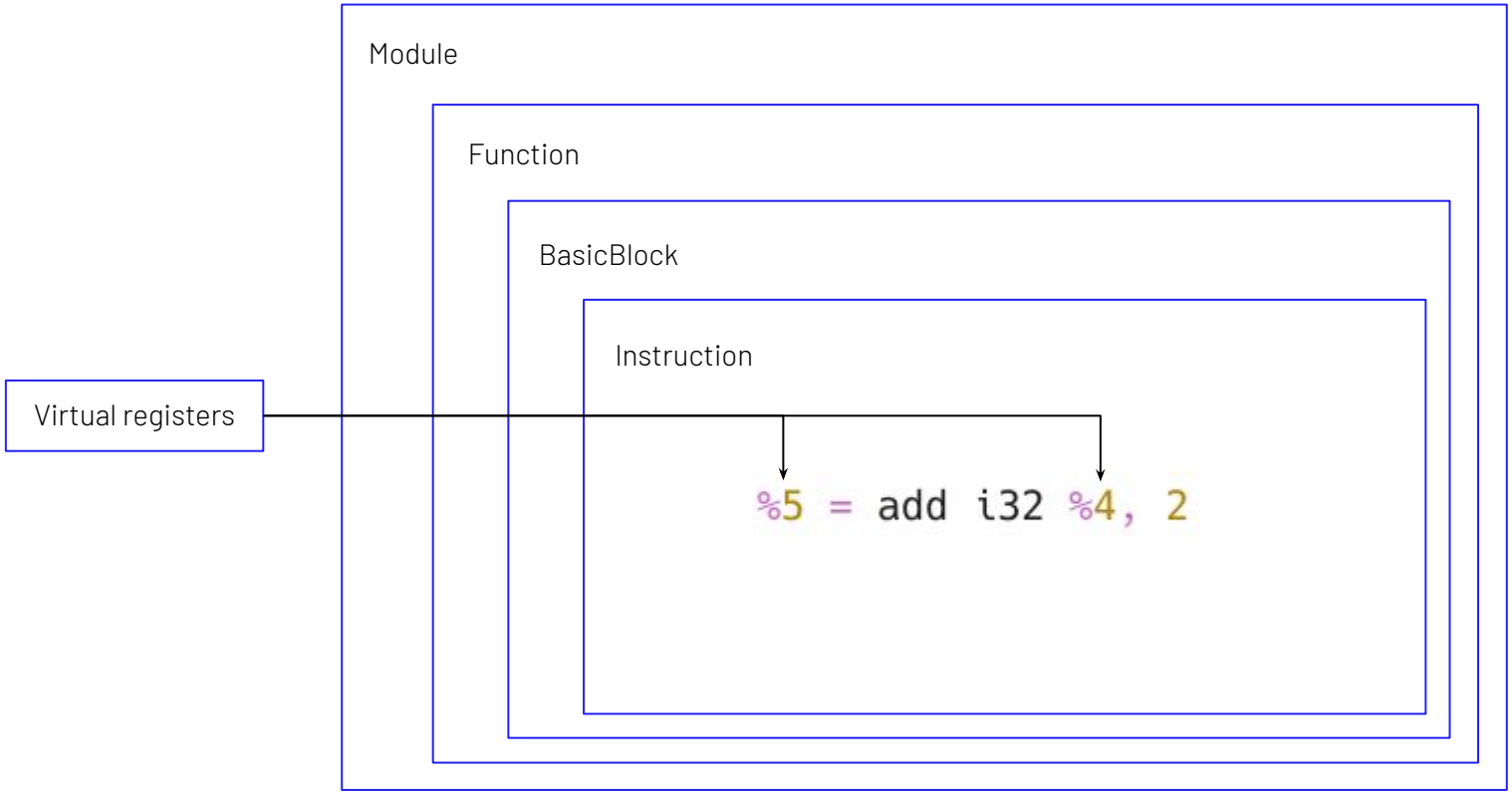
Into the IR

Lets go deeper



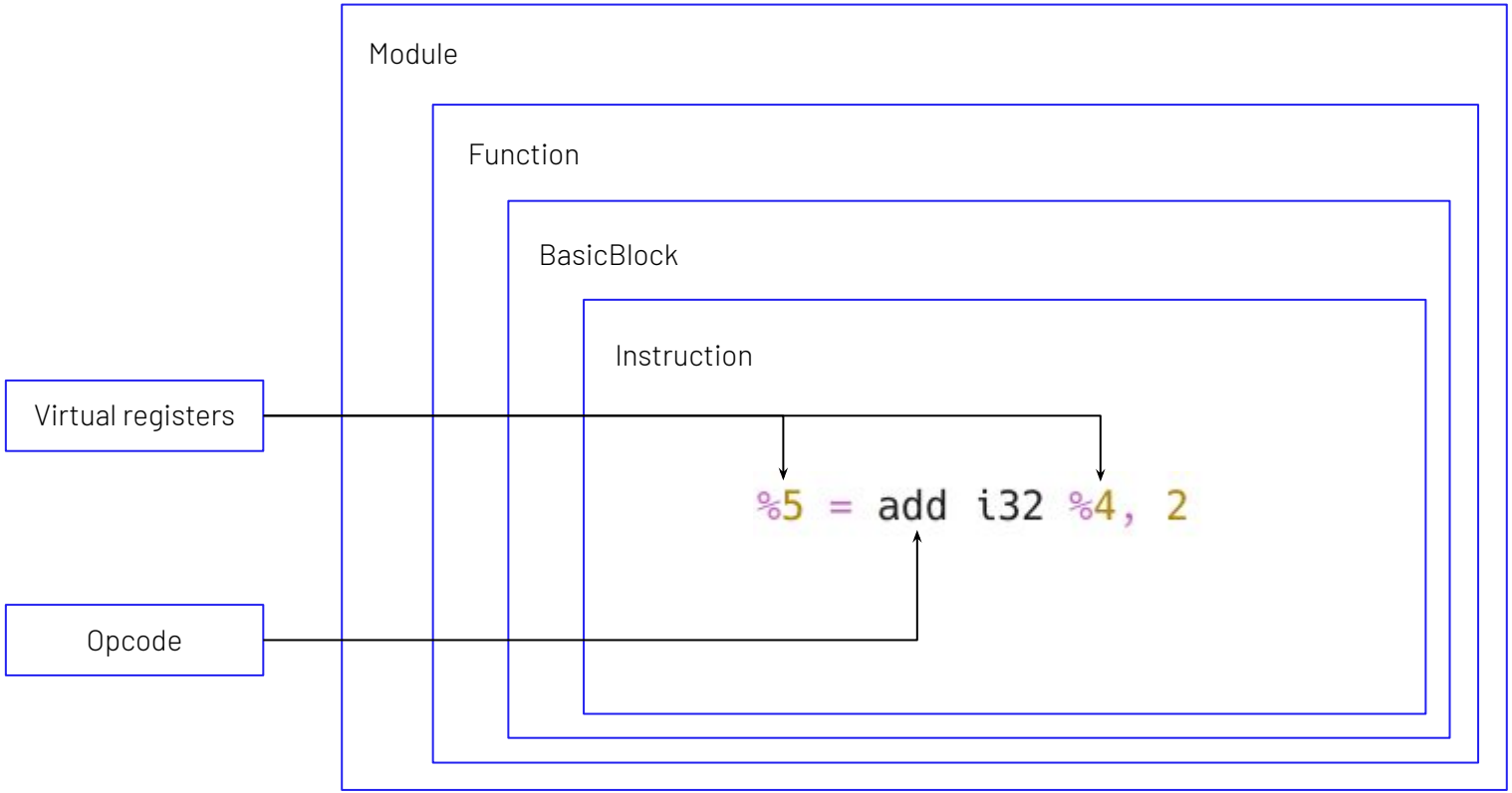
Into the IR

Lets go deeper



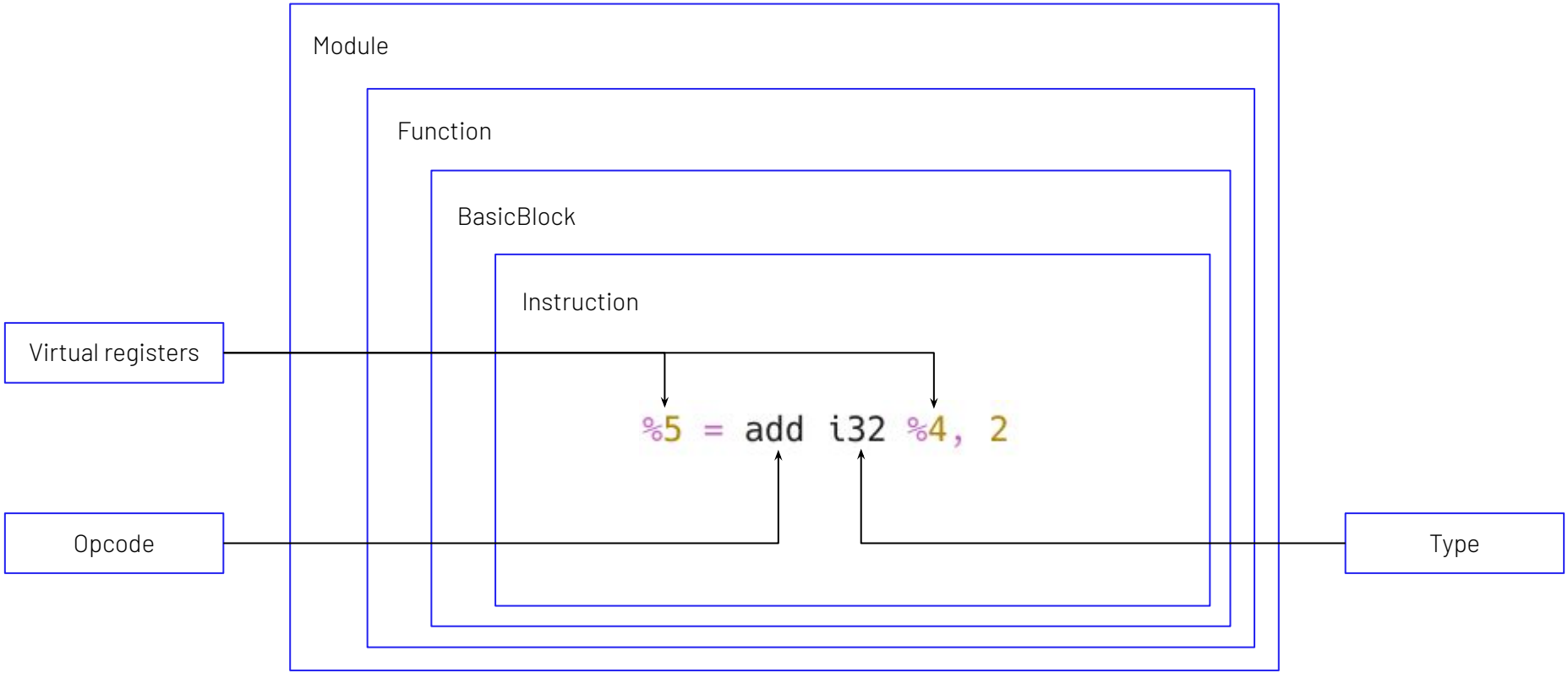
Into the IR

Lets go deeper



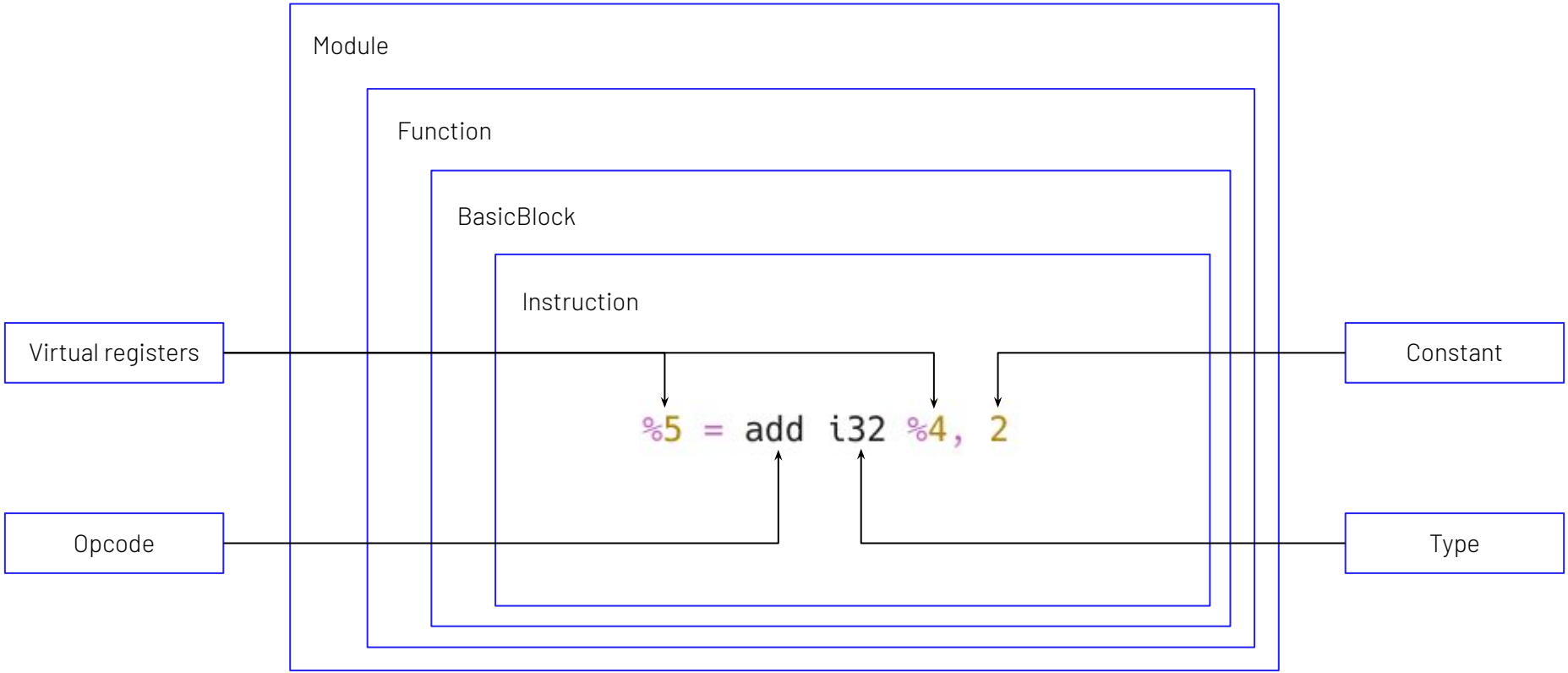
Into the IR

Lets go deeper



Into the IR

Lets go deeper



Listing function names

Lets print! (or cout, cuz cpp)

```
namespace {
    struct ListFunctionNames : public PassInfoMixin<ListFunctionNames> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            for (auto &F : M) {
                errs() << "Function name: " << F.getName() << "\n";
            }
            return PreservedAnalyses::all();
        };
    };
}
```

Listing function bodies

Lets print! (or cout, cuz cpp)

```
namespace {
    struct ListFunctions : public PassInfoMixin<ListFunctions> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            for (auto &F : M) {
                errs() << "Function:\n" << F << "\n";
            }
            return PreservedAnalyses::all();
        };
    };
}
```

Listing basic blocks

Lets print! (or cout, cuz cpp)

```
namespace {
    struct ListBasicBlocks : public PassInfoMixin<ListBasicBlocks> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            for (auto& F : M) {
                for (auto& BB : F) {
                    errs() << "Basic Block:\n" << BB << "\n";
                }
            }
            return PreservedAnalyses::all();
        };
    };
}
```

Listing instructions

Lets print! (or cout, cuz cpp)

```
namespace {
    struct ListInstructions : public PassInfoMixin<ListInstructions> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            for (auto& F : M) {
                for (auto& BB : F) {
                    for (auto& I : BB) {
                        errs() << "Instruction:\n" << I << "\n";
                    }
                }
            }
            return PreservedAnalyses::all();
        };
    };
}
```

Modifying program behaviour

Our first smol mod :D

Modifying program behaviour

Our first smol mod :D

```
extern "C" int add(int a, int b) {
    return a + b;
}
```

Modifying program behaviour

Our first smol mod :D

```
extern "C" int add(int a, int b) {
    return a + b;
}
```



```
extern "C" int add(int a, int b) {
    return a - b;
}
```

Modifying program behaviour

Our first smol mod :D

Modifying program behaviour

Our first smol mod :D

```
namespace {
    struct SimpleMod : public PassInfoMixin<SimpleMod> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            for (auto& F : M) {
                for (auto& BB : F) {
                    for (auto& I : BB) {
                        if (auto *binOp = dyn_cast<BinaryOperator>(&I)) {
                            if (binOp->getOpcode() == Instruction::Add) {
                                errs() << "Found add instruction: " << *binOp << "\n";

                                IRBuilder<> builder(binOp);
                                Value *newLHS = binOp->getOperand(0);
                                Value *newRHS = binOp->getOperand(1);
                                Value *newSub = builder.CreateSub(newLHS, newRHS);

                                binOp->replaceAllUsesWith(newSub);
                                errs() << "Replaced with sub instruction: " << *newSub << "\n";
                            }
                        }
                    }
                }
            }
            return PreservedAnalyses::none();
        };
    };
}
```

Modifying program behaviour

Our first smol mod :D

```
namespace {
    struct SimpleMod : public PassInfoMixin<SimpleMod> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            for (auto& F : M) {
                for (auto& BB : F) {
                    for (auto& I : BB) {
                        if (auto *binOp = dyn_cast<BinaryOperator>(&I)) {
                            if (binOp->getOpcode() == Instruction::Add) {
                                errs() << "Found add instruction: " << *binOp << "\n";

                                IRBuilder<> builder(binOp);
                                Value *newLHS = binOp->getOperand(0);
                                Value *newRHS = binOp->getOperand(1);
                                Value *newSub = builder.CreateSub(newLHS, newRHS);

                                binOp->replaceAllUsesWith(newSub);
                                errs() << "Replaced with sub instruction: " << *newSub << "\n";
                            }
                        }
                    }
                }
            }
            return PreservedAnalyses::none();
        };
    };
}
```

Check if instruction is an addition operation

Modifying program behaviour

Our first smol mod :D

```
namespace {
    struct SimpleMod : public PassInfoMixin<SimpleMod> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            for (auto& F : M) {
                for (auto& BB : F) {
                    for (auto& I : BB) {
                        if (auto *binOp = dyn_cast<BinaryOperator>(&I)) {
                            if (binOp->getOpcode() == Instruction::Add) {
                                errs() << "Found add instruction: " << *binOp << "\n";

                                IRBuilder<> builder(binOp);
                                Value *newLHS = binOp->getOperand(0);
                                Value *newRHS = binOp->getOperand(1);
                                Value *newSub = builder.CreateSub(newLHS, newRHS);

                                binOp->replaceAllUsesWith(newSub);
                                errs() << "Replaced with sub instruction: " << *newSub << "\n";
                            }
                        }
                    }
                }
            }
            return PreservedAnalyses::none();
        };
    };
}
```

Create new operation

Check if instruction is an addition operation

Modifying program behaviour

Our first smol mod :D

```
namespace {
    struct SimpleMod : public PassInfoMixin<SimpleMod> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            for (auto& F : M) {
                for (auto& BB : F) {
                    for (auto& I : BB) {
                        if (auto *binOp = dyn_cast<BinaryOperator>(&I)) {
                            if (binOp->getOpcode() == Instruction::Add) {
                                errs() << "Found add instruction: " << *binOp << "\n";

                                IRBuilder<> builder(binOp);
                                Value *newLHS = binOp->getOperand(0);
                                Value *newRHS = binOp->getOperand(1);
                                Value *newSub = builder.CreateSub(newLHS, newRHS);

                                binOp->replaceAllUsesWith(newSub);
                                errs() << "Replaced with sub instruction: " << *newSub << "\n";
                            }
                        }
                    }
                }
            }
            return PreservedAnalyses::none();
        };
    };
}
```

Check if instruction is an addition operation

Create new operation

Get Left and Right operands

Modifying program behaviour

Our first smol mod :D

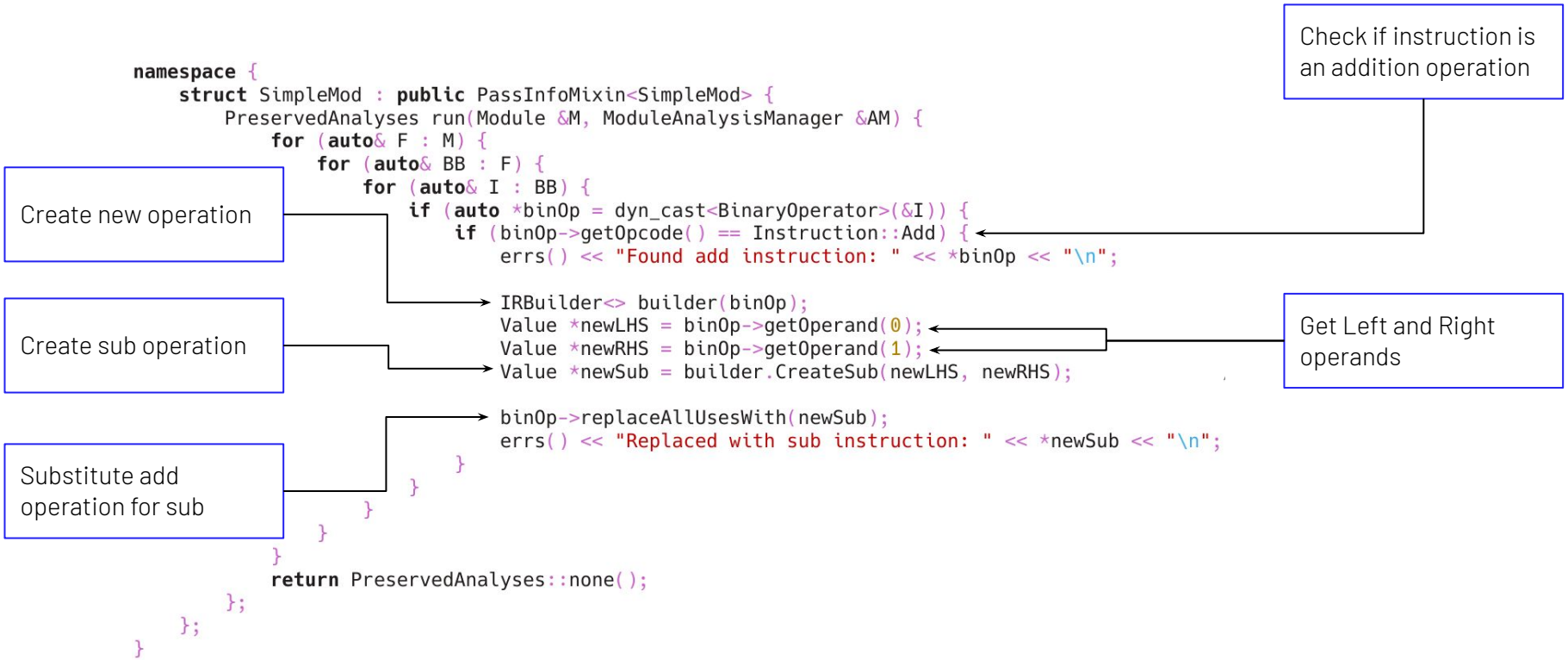

```
namespace {
    struct SimpleMod : public PassInfoMixin<SimpleMod> {
        PreservedAnalyses run(Module &M, ModuleAnalysisManager &AM) {
            for (auto& F : M) {
                for (auto& BB : F) {
                    for (auto& I : BB) {
                        if (auto *binOp = dyn_cast<BinaryOperator>(&I)) {
                            if (binOp->getOpcode() == Instruction::Add) {
                                errs() << "Found add instruction: " << *binOp << "\n";

                                IRBuilder<> builder(binOp);
                                Value *newLHS = binOp->getOperand(0);
                                Value *newRHS = binOp->getOperand(1);
                                Value *newSub = builder.CreateSub(newLHS, newRHS);

                                binOp->replaceAllUsesWith(newSub);
                                errs() << "Replaced with sub instruction: " << *newSub << "\n";
                            }
                        }
                    }
                }
            }
            return PreservedAnalyses::none();
        };
    };
}
```

Modifying program behaviour

Our first smol mod :D



Arithmetic Obfuscation

Let's make math mathing

Arithmetic Obfuscation

Let's make math mathing

Operation	MBA Transformation
X ^ Y	(X | Y) - (X & Y)
X + Y	(X & Y) + (X | Y)
X - Y	(X ^ -Y) + 2*(X & -Y)
X & Y	(X + Y) - (X | Y)
X | Y	X + Y + 1 + (~X | ~Y)

Arithmetic Obfuscation

Let's make math mathing

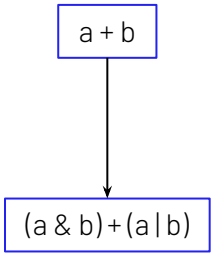
Operation	MBA Transformation
X ^ Y	(X | Y) - (X & Y)
X + Y	(X & Y) + (X | Y)
X - Y	(X ^ -Y) + 2*(X & -Y)
X & Y	(X + Y) - (X | Y)
X | Y	X + Y + 1 + (~X | ~Y)

a + b

Arithmetic Obfuscation

Let's make math mathing

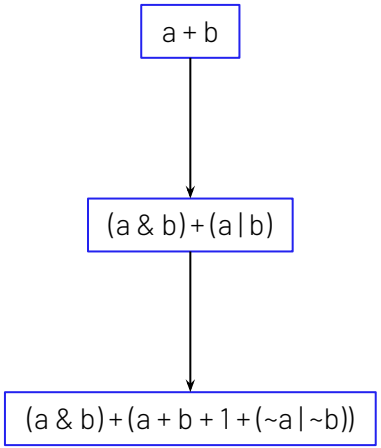
Operation	MBA Transformation
X ^ Y	(X | Y) - (X & Y)
X + Y	(X & Y) + (X | Y)
X - Y	(X ^ -Y) + 2*(X & -Y)
X & Y	(X + Y) - (X | Y)
X | Y	X + Y + 1 + (~X | ~Y)



Arithmetic Obfuscation

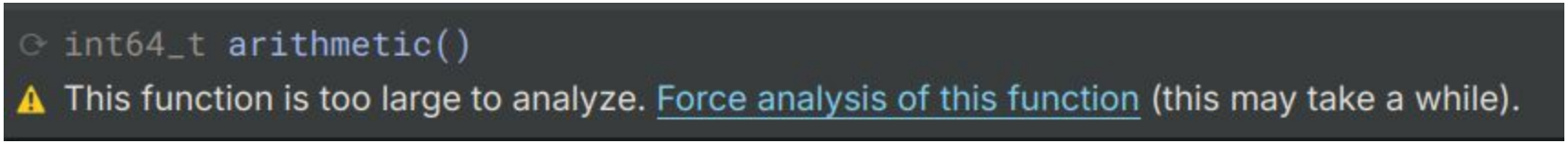
Let's make math mathing

Operation	MBA Transformation
X ^ Y	(X | Y) - (X & Y)
X + Y	(X & Y) + (X | Y)
X - Y	(X ^ -Y) + 2*(X & -Y)
X & Y	(X + Y) - (X | Y)
X | Y	X + Y + 1 + (~X | ~Y)



Arithmetic Obfuscation

Let's make math mathing



Split Basic Blocks

50% of the time, it works every time

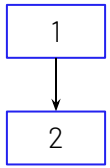
Split Basic Blocks

50% of the time, it works every time

1

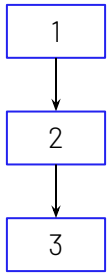
Split Basic Blocks

50% of the time, it works every time



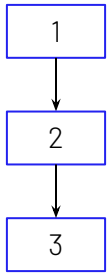
Split Basic Blocks

50% of the time, it works every time



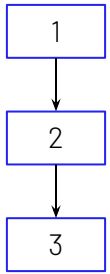
Split Basic Blocks

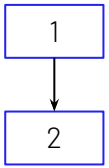
50% of the time, it works every time



Split Basic Blocks

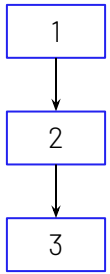
50% of the time, it works every time

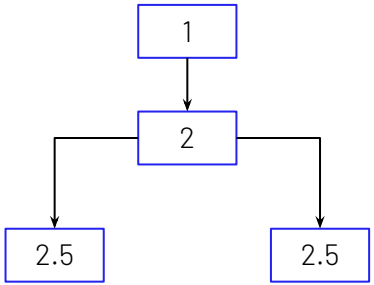




Split Basic Blocks

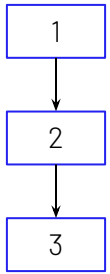
50% of the time, it works every time

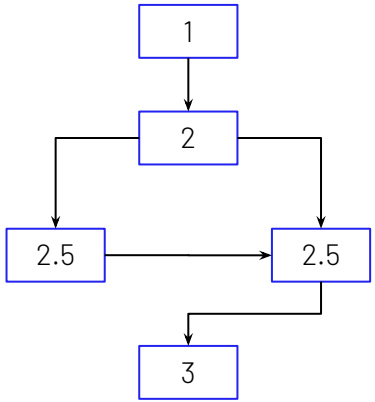




Split Basic Blocks

50% of the time, it works every time





Control Flow Flattening

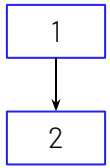
Let's get the bulldozer

Control Flow Flattening

Let's get the bulldozer

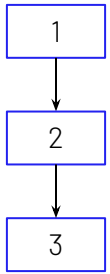
Control Flow Flattening

Let's get the bulldozer



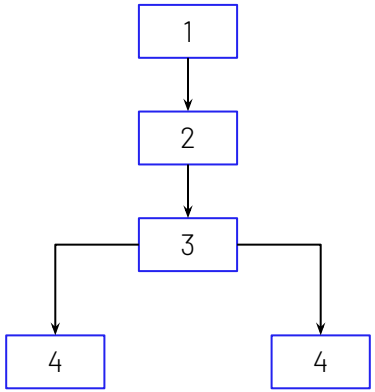
Control Flow Flattening

Let's get the bulldozer



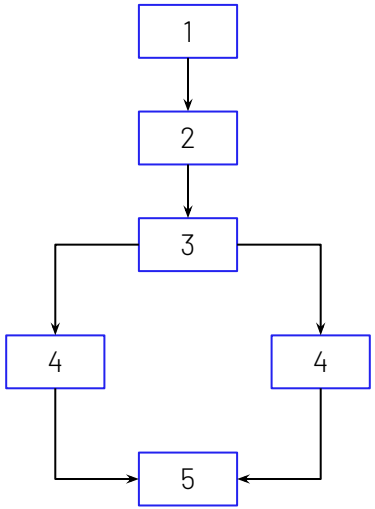
Control Flow Flattening

Let's get the bulldozer



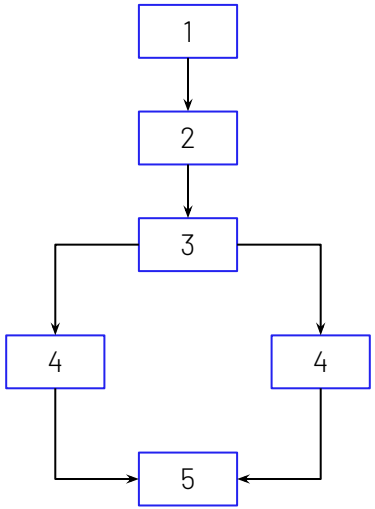
Control Flow Flattening

Let's get the bulldozer



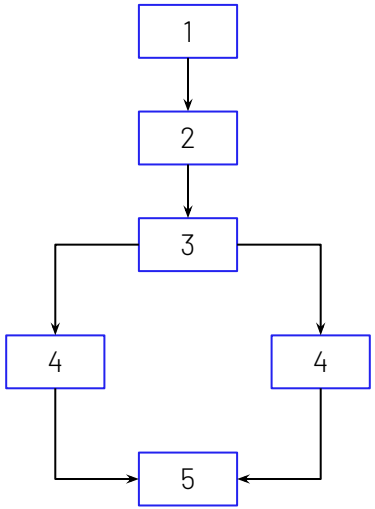
Control Flow Flattening

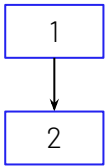
Let's get the bulldozer



Control Flow Flattening

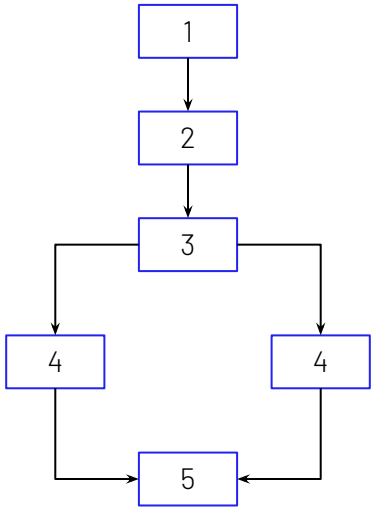
Let's get the bulldozer

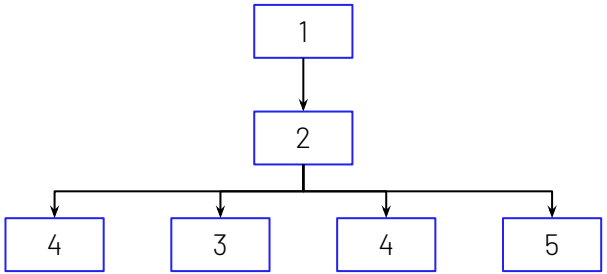




Control Flow Flattening

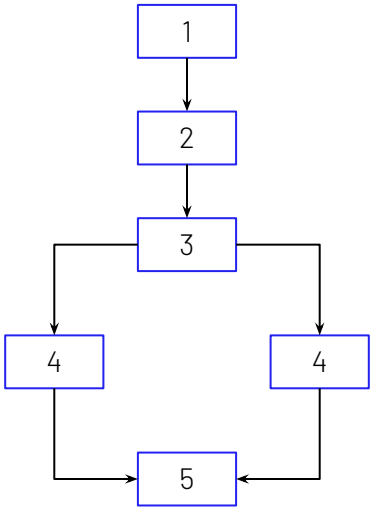
Let's get the bulldozer

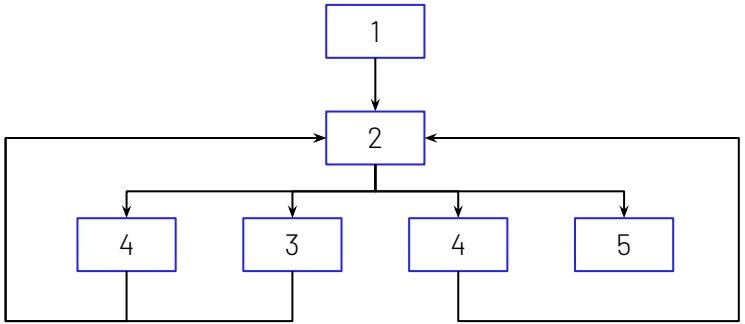




Control Flow Flattening

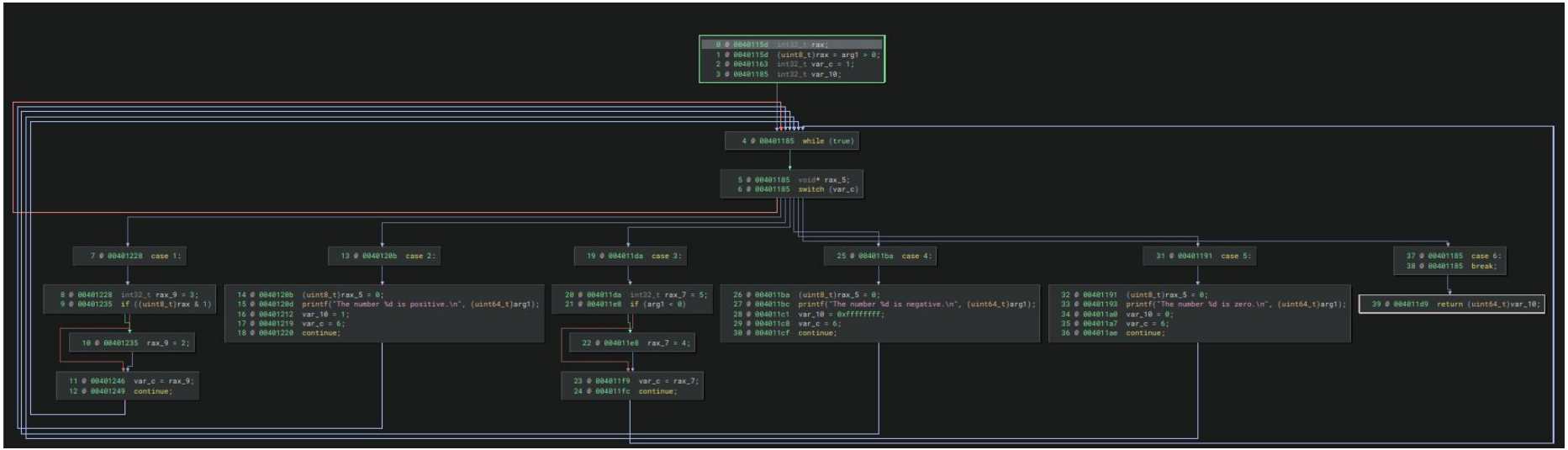
Let's get the bulldozer





Control Flow Flattening

Let's get the bulldozer



Obfuscation pipeline

It’s not a pipedream

Obfuscation pipeline

It's not a pipedream

```
#include "ControlFlowFlattening.cc"
#include "SplitBasicBlocks.cc"
#include "ArithmeticObf.cc"

extern "C" LLVM_ATTRIBUTE_WEAK PassPluginLibraryInfo llvmGetPassPluginInfo() {
    return {
        .APIVersion = LLVM_PLUGIN_API_VERSION,
        .PluginName = "Pipeline",
        .PluginVersion = "v0.1",
        .RegisterPassBuilderCallbacks = [](PassBuilder &PB) {
            PB.registerPipelineStartEPCallback(
                [](ModulePassManager &MPM, OptimizationLevel Level) {
                    MPM.addPass(ControlFlowFlattening());
                    MPM.addPass(SplitBasicBlocks());
                    MPM.addPass(ArithmeticObf());
                });
        }
    };
}
```

Obfuscation pipeline

It's not a pipedream

```
#include "ControlFlowFlattening.cc"
#include "SplitBasicBlocks.cc"
#include "ArithmeticObf.cc"

extern "C" LLVM_ATTRIBUTE_WEAK PassPluginLibraryInfo llvmGetPassPluginInfo() {
    return {
        .APIVersion = LLVM_PLUGIN_API_VERSION,
        .PluginName = "Pipeline",
        .PluginVersion = "v0.1",
        .RegisterPassBuilderCallbacks = [](PassBuilder &PB) {
            PB.registerPipelineStartEPCallback(
                [](ModulePassManager &MPM, OptimizationLevel Level) {
                    MPM.addPass(ControlFlowFlattening());
                    MPM.addPass(SplitBasicBlocks());
                    MPM.addPass(ArithmeticObf());
                });
        }
    };
}
```

Reference the passes

Obfuscation pipeline

It's not a pipedream

```
#include "ControlFlowFlattening.cc"
#include "SplitBasicBlocks.cc"
#include "ArithmeticObf.cc"

extern "C" LLVM_ATTRIBUTE_WEAK PassPluginLibraryInfo llvmGetPassPluginInfo() {
    return {
        .APIVersion = LLVM_PLUGIN_API_VERSION,
        .PluginName = "Pipeline",
        .PluginVersion = "v0.1",
        .RegisterPassBuilderCallbacks = [](PassBuilder &PB) {
            PB.registerPipelineStartEPCallback(
                [](ModulePassManager &MPM, OptimizationLevel Level) {
                    MPM.addPass(ControlFlowFlattening());
                    MPM.addPass(SplitBasicBlocks());
                    MPM.addPass(ArithmeticObf());
                });
        }
    };
}
```

Reference the passes

Add the passes by the order we want them to be executed

Have fun